

Вычислительная геометрия

Содержание

Вычислительная геометрия	1
1. Преобразование изображений. Геометрическое моделирование	2
1.1 Основные понятия	2
1.2 Модели линейных пространств	2
1.3 Геометрические преобразования	3
1.4 Линейные операторы	4
1.5 Аффинное преобразование	5
1.6 Однородные координаты	7
1.7 Моделирование плоских линий	10
1.7.1 Общие сведения	10
1.7.2 Дифференциальные характеристики	12
1.7.3 Конструирование кривой по Безье	14
1.7.4 Дискретизация (растровое изображение) линий	17
2. Геометрический поиск	20
2.1 Пересечение отрезков	20
2.2 Локализация точки в многоугольнике	24
2.2.1 Локализация точки в прямоугольнике	24
2.2.2 Локализация точки в n -угольнике	25
2.3 Выпуклые оболочки	28
2.4 Близость	31
2.4.1 Ближайшая пара	32
2.4.2 Построение локус-сети (диаграммы Вороного) множества	32

1. Преобразование изображений. Геометрическое моделирование

1.1 Основные понятия

Мет. Линейное пространство – это множество векторов V с определенными операциями сложения $+$ и умножения на число $\cdot \lambda$, где $\lambda \in \mathbb{R}(\mathbb{C})$, которые удовлетворяют свойствам:

1.-4. свойства абелево-аддитивной группы:

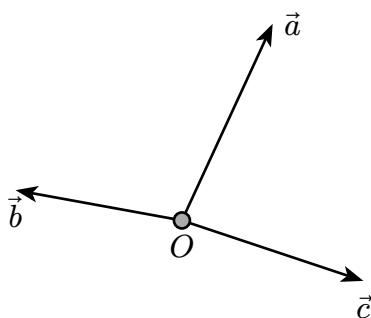
1. $x + y = y + x$ для $x, y \in V$
2. $x + (y + z) = (x + y) + z$ для $x, y, z \in V$
3. Существует такой 0 , что $x + 0 = x$ для $x \in V$
4. Для любого $x \in V$ существует такой $-x$, что $x + (-x) = 0$
5. $1 \cdot x = x \cdot 1 = x$ для $x \in V$
6. $(\lambda\mu)x = \lambda(\mu x)$ для $x \in V$ и $\lambda, \mu \in \mathbb{R}(\mathbb{C})$
7. $(\lambda + \mu)x = \lambda x + \mu x$ для $x \in V$ и $\lambda, \mu \in \mathbb{R}(\mathbb{C})$
8. $\lambda(x + y) = \lambda x + \lambda y$ для $x, y \in V$ и $\lambda \in \mathbb{R}(\mathbb{C})$

В общем случае умножение определено на комплексное число, но мы будем рассматривать вещественные

Def. Линейная комбинация векторов $x, y, z, \dots$ называется сумма $\lambda_1 x + \lambda_2 y + \lambda_3 z + \dots$, где $\lambda_i \in \mathbb{R}(\mathbb{C})$

1.2 Модели линейных пространств

Геометрическое



Линейное пространство – направленные отрезки с общим началом

Арифметическое

$$x = (x_1, x_2) \text{ в } \mathbb{R}^2$$

$$x = (x_1, x_2, x_3) \text{ в } \mathbb{R}^3$$

$$\text{В общем случае } x = (x_1, \dots, x_n) \text{ в } \mathbb{R}^n$$

Линейное пространство – множество упорядоченных совокупностей n чисел

Между этими моделями вводится изоморфизм с помощью базиса, например, $(\vec{i}, \vec{j}, \vec{k})$. Тогда всякий геометрический вектор можно преобразовать в арифметический и наоборот: $\vec{x} = x_1\vec{i} + x_2\vec{j} = (x_1, x_2)$

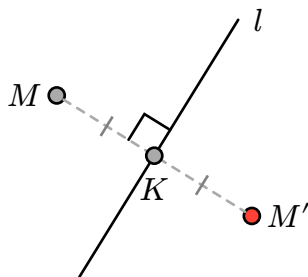
1.3 Геометрические преобразования

Def. Геометрическое преобразование – это биекция, которая переводит пространство Ω в себя

Def. Движение – геометрическое преобразование, сохраняющее расстояние между двумя любыми точками (изометрия)

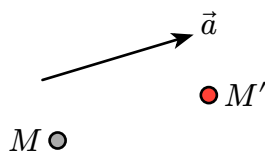
Виды движения на плоскости $\mathbb{R}^2$:

1. Осевая симметрия S_l относительно оси l



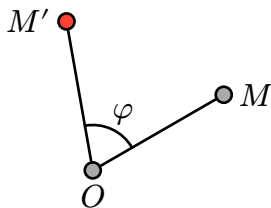
$$M' = S_l(M) \text{ так, что } \begin{cases} MM' \perp l \\ MK = KM' \end{cases}$$

2. Перенос $T_{\vec{a}}$ на вектор $\vec{a}$



$$M' = T_{\vec{a}}(M) \text{ так, что } \overrightarrow{MM'} = \vec{a}$$

3. Поворот R_O^φ относительно точки O на ориентированный угол φ



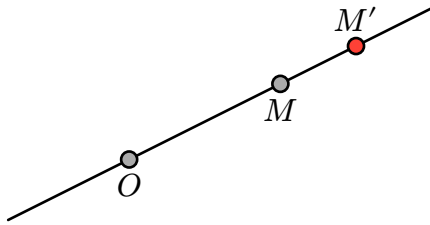
$$M' = R_O^\varphi(M) \text{ так, что } \begin{cases} \angle(MOM') = \varphi \\ OM = OM' \end{cases}$$

Традиционно принимаем положительный угол за поворот против часовой стрелки

Def. Конформное преобразование – преобразование, сохраняющее углы

Виды конформных преобразований на плоскости $\mathbb{R}^2$:

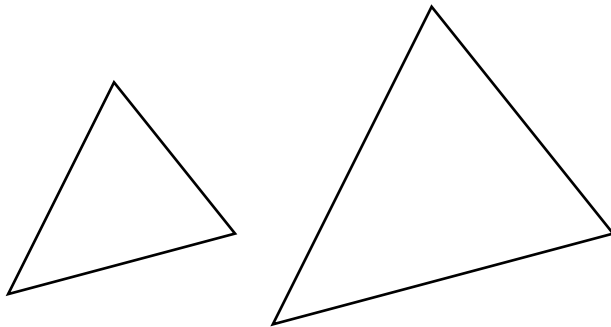
1. Гомотетия H_O^k относительно точки O с коэффициентом $k \in \mathbb{R}$



$$M' = H_O^k(M) \text{ так, что } \begin{cases} M' \in OM \\ \frac{OM'}{OM} = k \end{cases}$$

Nota. Если $k < 0$, то точки M и M' будут по разные стороны от точки O

2. Подобие P_k с коэффициентом k – композиция движения и гомотетии $P_k = F \circ H_O^k$ (здесь $(f \circ g)(x) = f(g(x))$)



1.4 Линейные операторы

Def. Линейный оператор $\mathcal{A}$ – отображение $\mathcal{A} : V \rightarrow V$ (в общем случае $\mathcal{A} : V \rightarrow W$), для которого соблюдаются свойства:

1. $\mathcal{A}(x + y) = \mathcal{A}(x) + \mathcal{A}(y)$
2. $\mathcal{A}(\lambda x) = \lambda \mathcal{A}(x)$

Если для V определен базис $\varepsilon_V = \{e_1, \dots, e_n\}$, а для W базис $\varepsilon_W = \{e'_1, \dots, e'_m\}$, то действие оператора можно представить так:

$$\mathcal{A}(x) = \mathcal{A}\left(\sum_{i=1}^n \lambda_i e_i\right) = \sum_{i=1}^n \lambda_i \mathcal{A}(e_i) = \sum_{i=1}^n \lambda_i \sum_{j=1}^m \mu_j e'_j = \sum_{i=1}^n \sum_{j=1}^m a_{ij} e'_j$$

Def. Матрица $A = \{a_{ij}\}_{i=1..m, j=1..n} = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix}$ называется матрицей оператора

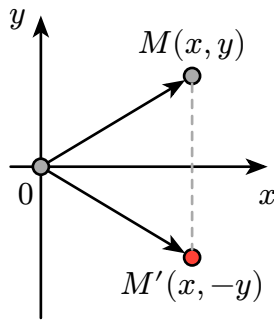
$\mathcal{A} : V \rightarrow W$

$$\text{Тогда } \mathcal{A}x = y \iff A \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix}$$

Найдем матрицы геометрических преобразований на $\mathbb{R}^2$

1. Осевая симметрия S_l

Чаще всего на практике используются S_{Ox} и S_{Oy}



Для S_{Ox} это матрица $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$: $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x \\ -y \end{pmatrix}$

А для Oy это $\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$: $\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} -x \\ y \end{pmatrix}$

2. Перенос $T_{\vec{a}}$ на вектор

Перенос точки на вектор выносит ее из линейного пространства, где точки имеют общее начало, поэтому перенос не является линейным оператором: $\begin{pmatrix} x \\ y \end{pmatrix} +$

$$\begin{pmatrix} a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} x' \\ y' \end{pmatrix}$$

3. Поворот

Активным преобразованием называется поворот плоскости, а пассивным – поворот системы координат. Такие преобразования взаимно обратны

Тогда поворот системы координат задается матрицей $\begin{pmatrix} \cos \varphi & \sin \varphi \\ -\sin \varphi & \cos \varphi \end{pmatrix}$

А поворот плоскости задается обратной матрицей $\begin{pmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{pmatrix}$

4. Гомотетия

Для гомотетии H_O^k матрица оператора равна $\begin{pmatrix} k & 0 \\ 0 & k \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x' \\ y' \end{pmatrix}$

1.5 Аффинное преобразование

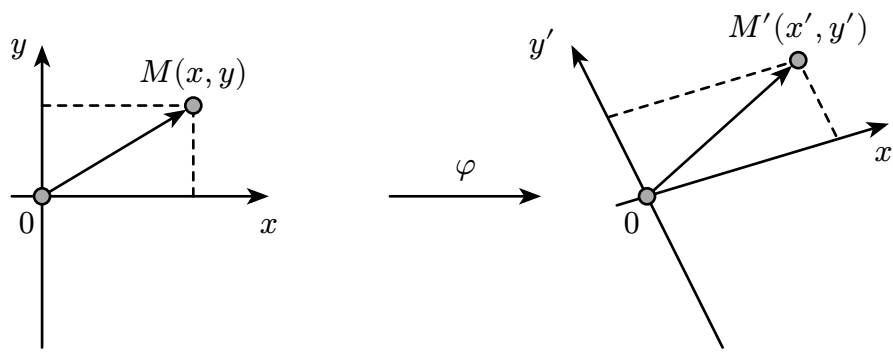
Def. 1. Преобразование $\varphi: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ называется аффинным преобразованием, если φ – биекция, и для всяких точек на прямой $A, B, C \in l$ справедливо, что

$$\varphi(A), \varphi(B), \varphi(C) \in \varphi(l) \text{ и } \frac{AB}{BC} = \frac{\varphi(A)\varphi(B)}{\varphi(B)\varphi(C)}$$

Nota. Аффинное преобразование не сохраняет углы и расстояния, но сохраняет параллельность

Nota. Кроме этого все треугольники аффинно эквивалентны, то есть один треугольник можно перевести в любой другой с помощью аффинного преобразования

Def. 2. Преобразование φ – аффинное преобразование, если оно переводит одну систему координат в другую систему координат



Мет. Система координат – это определенная точка отсчета, координатная сетка, порядок осей и единичные отрезки

Для дальнейшего нам потребуются уравнения прямых:

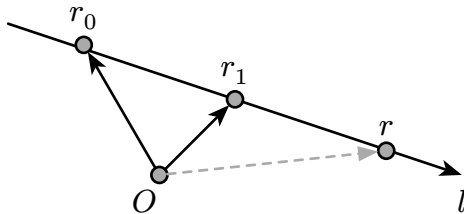
1. По двум точкам на плоскости:

$$\frac{x - x_A}{x_B - x_A} = \frac{y - y_A}{y_B - y_A} \Leftrightarrow \begin{vmatrix} x - x_A & y - y_A \\ x_B - x_A & y_B - y_A \end{vmatrix} = 0$$

2. По коэффициентам на плоскости:

$$\begin{cases} x = mt + x_0 \\ y = nt + y_0 \end{cases}$$

3. Векторное: $\vec{r} - \vec{r}_0 = \vec{a}t$, где t – параметр



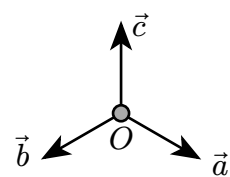
$$\vec{r} - \vec{r}_0 = (\vec{r}_1 - \vec{r}_0)t$$

Также нам понадобятся:

- Уравнения плоскостей в пространстве
- Уравнения кривых второго порядка, специальных кривых (спирали, гипоциклоиды)
- Индикатор ориентации

Мет. $\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + \dots + a_n b_n = |\vec{a}| |\vec{b}| \cos(\vec{a}, \vec{b})$

$$\vec{a} \times \vec{b} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = \vec{c} : \begin{cases} |\vec{c}| = |\vec{a}| |\vec{b}| \sin(\widehat{\vec{a}, \vec{b}}) \\ \vec{c} \perp \vec{a} \\ \vec{c} \perp \vec{b} \\ (\vec{a}, \vec{b}, \vec{c}) - \text{правая тройка векторов} \end{cases}$$

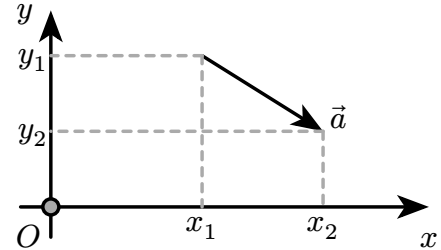


Def. Псевдоскалярное (или косое) произведение $\vec{a} \vee \vec{b} \stackrel{\text{def}}{=} |\vec{a}| |\vec{b}| \sin(\vec{a}, \vec{b})$, причем произведение положительно, если угол между $\vec{a}$ и $\vec{b}$ положителен (то есть против часовой), и отрицательно, если угол отрицателен (то есть по часовой)

1.6 Однородные координаты

Мет. В плоскости $\mathbb{R}^2$ существует линейное пространство направленных отрезков. Проблема состоит в том, что нам нужно представить вектор с другим началом

Тогда такие вектора можно представить двумя точками



Чтобы работать с точками, а не векторами с общим началом O , обобщим понятие линейного пространства. Тогда понятие линейного пространства обобщается до аффинного, где элементы – это точки, а не векторы

Def. Пространство $\mathcal{A}$ – аффинное пространство, ассоциированное с линейным пространством V , если:

1. Заданы аксиомы для V
2. Существует $f : \mathcal{A} \rightarrow V$ такое, что для всякой пары сопоставляется вектор из линейного: $\forall A, B \in \mathcal{A} \quad f(A, B) = \overrightarrow{AB} \in V$
3. Для всяких $A \in \mathcal{A}$ и $\vec{a} \in V$ существует единственная $B \in \mathcal{A} \mid \overrightarrow{AB} = \vec{a}$
4. Для всяких точек $A, B, C \in \mathcal{A}$ справедливо, что $\overrightarrow{AB} + \overrightarrow{BC} = \overrightarrow{AC}$

В аффинном пространстве $\mathcal{A}$ можно ввести аффинные преобразования. Те, что не связаны с переносом, можно считать линейными в пространстве V :

1. Осевая симметрия S_l , если $O \in l$
2. Поворот R_O^φ
3. Гомотетия H_O^k

Их можно представить в виде матрицы. Но перенос выводит из линейного пространства. Нам нужно все преобразования свести к алгебраическому действию $x' = \mathcal{F}x$, где $\mathcal{F}$ – преобразование с матрицей F

Движение плоскости и гомотетия дают формулу:

$$X' = FX + T_{\vec{a}}$$

Вместо

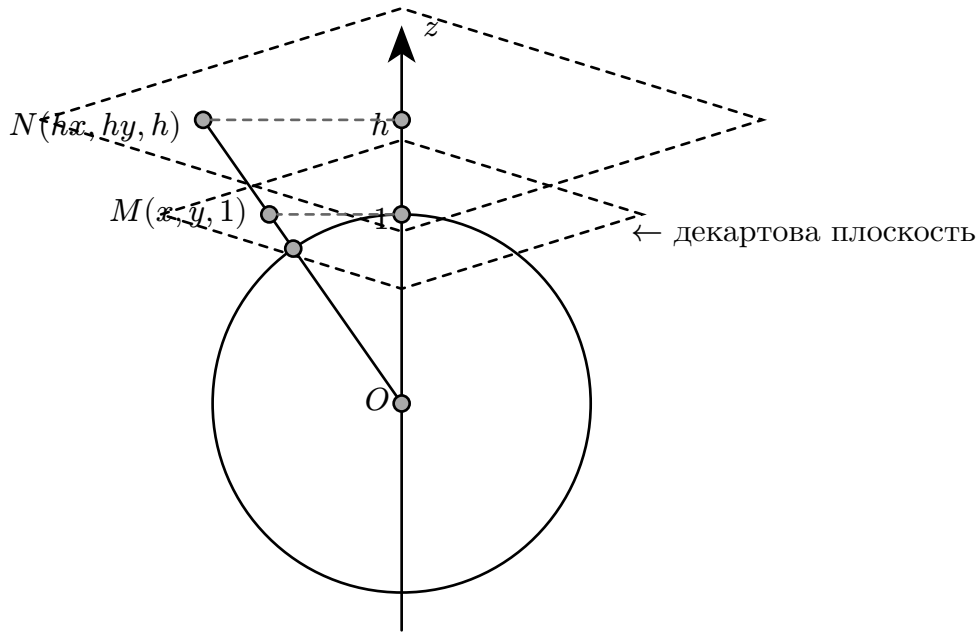
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} f_{11} & f_{12} \\ f_{21} & f_{22} \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} x_0 \\ y_0 \end{pmatrix},$$

рассмотрим векторы с добавленной координатой $z = 1$:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} f_{11} & f_{12} & x_0 \\ f_{21} & f_{22} & y_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Тогда $\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} f_{11}x + f_{12}y + x_0 \\ f_{21}x + f_{22}y + y_0 \\ 1 \end{pmatrix}$. Координаты $\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix}$ и $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$ называют однородными

Геометрическая интерпретация – стереографическая проекция Римана



Далее происходит центральное проектирование на плоскость $z = h, p(x, y) \rightarrow N(hx, hy, h)$

Таким образом, каждой точке декартовой плоскости ставится в соответствии точка сферы, а она центрально проектируется на плоскость $z = h$, где h отвечает за масштаб. В результате точкам декартовой плоскости (x, y) соответствуют точки $(x, y, 1) = (hx, hy, h)$, а однородные координаты $(x, y, 0)$ представляют бесконечно удаленную точку декартовой плоскости в направлении вектора $\vec{a} = (x, y)$

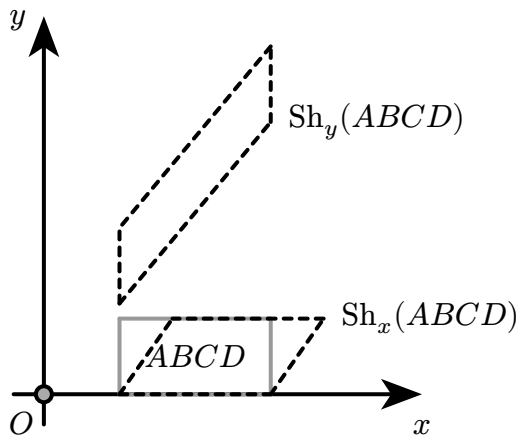
Рассмотрим матрицы преобразований в однородных координатах:

$$F = \begin{pmatrix} a & c & m \\ b & d & h \\ p & q & s \end{pmatrix}$$

$\begin{pmatrix} a & c \\ b & d \end{pmatrix}$ представляет композицию из симметрии, поворота, гомотетии и сдвига

Def. Сдвиг (shear) Sh_x – наклонной перекося такой, что $\begin{cases} x' = x + ky \\ y' = y \end{cases}$, а $k = sh_x$

Аналогично по оси Oy сдвиг $\begin{cases} x' = x \\ y' = y + sh_y x \end{cases}$



Вектор $\begin{pmatrix} m \\ n \end{pmatrix}$ – вектор переноса $T_{(m,n)}$, а число s отвечает за общее масштабирование

Рассмотрим смысл (p, q) :

$$\begin{pmatrix} 1 & 0 & | & 0 \\ 0 & 1 & | & 0 \\ \hline p & q & | & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ px + qy + 1 \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ h \end{pmatrix}$$

При фиксированных p и q выражение $h = px + qy + 1$ задает наклонную плоскость в трехмерном пространстве, что позволяет изменять перспективу. На этом курсе операции, использующие p и q , рассматриваться не будут

Рассмотрим частные виды преобразований:

- Перенос $T_{m,n} = \begin{pmatrix} 1 & 0 & | & m \\ 0 & 1 & | & n \\ \hline 0 & 0 & | & 1 \end{pmatrix}$
- Поворот $R_O^\varphi = \begin{pmatrix} \cos \varphi & \sin \varphi & | & 0 \\ -\sin \varphi & \cos \varphi & | & 0 \\ \hline 0 & 0 & | & 1 \end{pmatrix}$
- Симметрия по оси $S_{Ox} = \begin{pmatrix} 1 & 0 & | & 0 \\ 0 & -1 & | & 0 \\ \hline 0 & 0 & | & 1 \end{pmatrix}$
- Симметрия по биссектрисе $S_{x=y} = \begin{pmatrix} 0 & 1 & | & 0 \\ 1 & 0 & | & 0 \\ \hline 0 & 0 & | & 1 \end{pmatrix}$
- Сдвиг $Sh_x = \begin{pmatrix} 1 & sh_x & | & 0 \\ 0 & 1 & | & 0 \\ \hline 0 & 0 & | & 1 \end{pmatrix}$

Ех. Дан $\triangle ABC$ с вершинами в координатах $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$. Найти $\triangle A'B'C' = \mathcal{F}(\triangle ABC)$

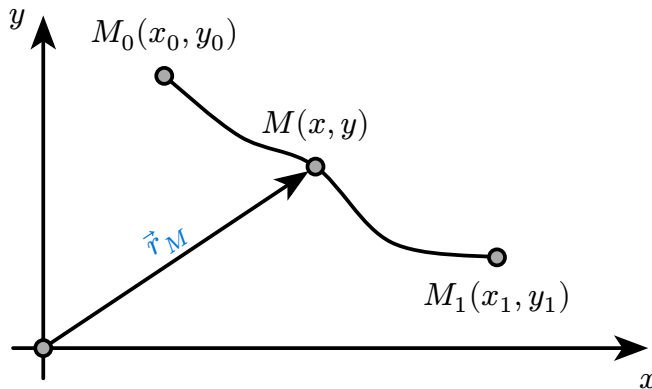
Найдем матрицу координат вершин $\triangle ABC$: $\begin{pmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{pmatrix}$

Преобразование осуществляется так:

$$\begin{pmatrix} a & c & m \\ b & d & h \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} x'_1 & x'_2 & x'_3 \\ y'_1 & y'_2 & y'_3 \\ 1 & 1 & 1 \end{pmatrix}$$

1.7 Моделирование плоских линий

1.7.1 Общие сведения



Пусть $\{\vec{e}_1, \vec{e}_2\}$ – базис (в частности $\{\vec{i}, \vec{j}\}$ – декартов)

Радиус-вектор точки кривой γ определяется как $\vec{r}_M = x\vec{e}_1 + y\vec{e}_2$

Чаще всего кривая γ ориентирована так, что задана начальная точка $M_0(x_0, y_0)$ и пара уравнений $\begin{cases} x = x(t) \\ y = y(t) \end{cases}$, где $t \in [t_1, t_2]$. Интервал $[t_1, t_2]$ задают ориентацию

Таким образом, γ может быть задана:

- параметрически $\vec{r}(t) = x(t)\vec{e}_1 + y(t)\vec{e}_2$
- общим уравнением $f(x, y) = 0$

Рассмотрим задания простых кривых:

1. Прямая

$$\vec{r} = \vec{r}_0 + t(\vec{r}_1 - \vec{r}_0) = t\vec{r}_1 + \vec{r}_0(1 - t)$$

Последнее выражение полезно, так как для $t \in [0, 1]$ уравнение задает отрезок M_0M_1 : $\vec{r}(t)|_{t=0} = \vec{r}_0$, $\vec{r}(t)|_{t=1} = \vec{r}_1$

2. Окружность

Параметрическое уравнение: $\begin{cases} x = R \cos t \\ y = R \sin t \end{cases}$ для $t \in [0, 2\pi)$

Радиус-вектор задается как $\vec{r}(t) = R(\vec{i} \cos t + \vec{j} \sin t) = R(\cos t, \sin t)$

3. Кривая второго порядка в общем виде

$$a_{11}x^2 + 2a_{12}xy + a_{22}y^2 + 2a_{13}x + 2a_{23}y + a_{33} = 0$$

При этом один или более из коэффициентов a_{11}, a_{12}, a_{22} не равны нулю

Коэффициенты кривой можно представить в виде матрицы: $A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{pmatrix}$

Коэффициенты a_{13} и a_{23} отвечают за перенос кривой, а a_{33} за масштаб

В однородных координатах $\vec{r} = (x, y) \rightarrow (x, y, 1)$ получаем:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11}x + a_{12}y + a_{13} \\ a_{12}x + a_{22}y + a_{23} \\ a_{13}x + a_{23}y + a_{33} \end{pmatrix}$$

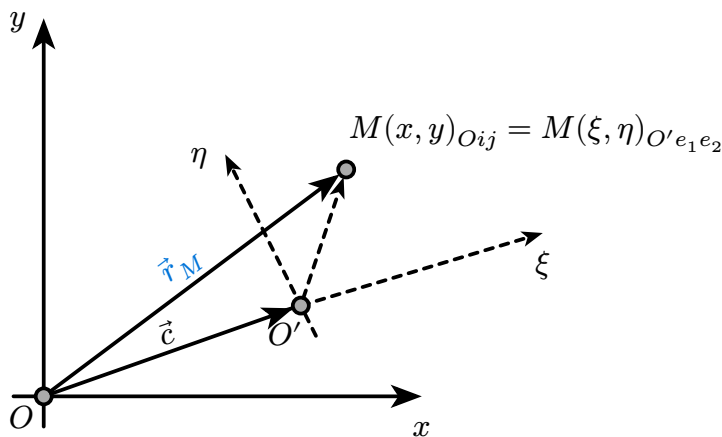
Если слева домножить на вектор-строку $(x, y, 1)$, то получим:

$$\begin{aligned} (x, y, 1) \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} &= \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}^T \begin{pmatrix} a_{11}x + a_{12}y + a_{13} \\ a_{12}x + a_{22}y + a_{23} \\ a_{13}x + a_{23}y + a_{33} \end{pmatrix} \\ &= a_{11}x^2 + a_{12}xy + a_{13}x + a_{12}xy + a_{22}y^2 + a_{23}y + a_{13}x + a_{23}y + a_{33} \\ &= a_{11}x^2 + 2a_{12}xy + a_{22}y^2 + 2a_{13}x + 2a_{23}y + a_{33} \end{aligned}$$

Тогда общее уравнение кривой второго порядка в матричном виде записывается

$$\text{как } (x, y, 1) \cdot A \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = 0$$

Nota. Пусть есть аффинный базис $\{\vec{e}_1, \vec{e}_2\}$. Нужно перевести координаты точки $M(\xi, \eta)$ из нового базиса $\{\vec{e}_1, \vec{e}_2\}$ в координаты $\begin{pmatrix} x \\ y \end{pmatrix}$ из нового базиса $\{\vec{i}, \vec{j}\}$



Радиус-вектор точки в базисе $\begin{pmatrix} x \\ y \end{pmatrix}$ равен $\vec{r}_M = \begin{pmatrix} x \\ y \end{pmatrix} = \xi \vec{e}_1 + \eta \vec{e}_2 + \vec{c}$. Здесь $\vec{c}$ — смещение начал координат

В однородных координатах это выглядит так:

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} \xi e_{1x} \\ \xi e_{1y} \\ 0 \end{pmatrix} + \begin{pmatrix} \eta e_{2x} \\ \eta e_{2y} \\ 0 \end{pmatrix} + \begin{pmatrix} c_1 \\ c_2 \\ 1 \end{pmatrix} = \begin{pmatrix} e_{1x} & e_{2x} & c_1 \\ e_{1y} & e_{2y} & c_2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \xi \\ \eta \\ 1 \end{pmatrix} = P \begin{pmatrix} \xi \\ \eta \\ 1 \end{pmatrix}$$

Здесь $P = \begin{pmatrix} e_{1x} & e_{2x} & c_1 \\ e_{1y} & e_{2y} & c_2 \\ 0 & 0 & 1 \end{pmatrix}$ – матрица аффинного преобразования, где $\begin{pmatrix} e_{1x} & e_{2x} \\ e_{1y} & e_{2y} \end{pmatrix}$ – матрица приведения из базиса $\{\vec{i}, \vec{j}\}$ в базис $\{\vec{e}_1, \vec{e}_2\}$

Такая матрица не меняет инварианты, поэтому можно делать подобные преобразования кривых:

$$(x, y, 1)A \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = 0 \implies (\xi, \eta, 1)PAP^{-1} \begin{pmatrix} \xi \\ \eta \\ 1 \end{pmatrix} = 0$$

Def. Инварианты кривой второго порядка – выражения, значения которых остаются постоянными при применении аффинных преобразований:

$$I_1 = a_{11} + a_{22}, I_2 = \begin{vmatrix} a_{11} & a_{12} \\ a_{12} & a_{22} \end{vmatrix}, I_3 = \begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{vmatrix}$$

Тип кривой определяется по инвариантам:

- Если $I_2 > 0$, то кривая эллиптического типа
- Если $I_2 < 0$, то кривая гиперболического типа
- Если $I_2 = 0$, то кривая параболического типа

1.7.2 Дифференциальные характеристики

Мет. Для кривой $\gamma : \begin{cases} x = x(t) \\ y = y(t) \end{cases}$

- Гладкость кривой – непрерывная дифференцируемость $x(t)$ и $y(t)$, то есть для всякой $M \in \gamma$ существуют $\frac{dy}{dx} = \varphi(t)$, которая непрерывная
- Касательная – вектор (или прямая), имеющая одну общую точку с кривой в окрестности

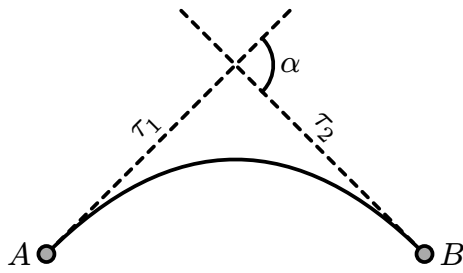
Если кривая задается как $\vec{r}(t) = x(t)\vec{i} + y(t)\vec{j}$, то касательная задается как производная:

$$\vec{r}'(t) = \frac{d\vec{r}(t)}{dt} = \frac{dx}{dt}\vec{i} + \frac{dy}{dt}\vec{j}$$

- Нормаль – перпендикуляр к кривой в точке. Вектор нормали задается как перпендикулярный к касательной: $\vec{n} \perp \vec{r}'$
- Длина дуги (элемента): $ds = \sqrt{x'^2(t) + y'^2(t)}dt = |\vec{r}'(t)| dt$

Рассмотрим новые характеристики для кривой:

- Кривизна кривой в точки



Касательная τ_1 переходит в τ_2 при $A \rightarrow B$, поворачиваясь на угол α – угол смежности

Средняя кривизна на дуге $\overset{\frown}{AB}$ равна $K_{\text{ср}} = \frac{\alpha}{|\overset{\frown}{AB}|}$

Def. Кривизна кривой γ в точке A определяется как $K_A = \lim_{\substack{B \rightarrow A \\ \text{по } AB}} K_{\text{ср}} = \lim_{|\overset{\frown}{AB}| \rightarrow 0} \frac{\alpha}{|\overset{\frown}{AB}|}$

Ex. Окружность

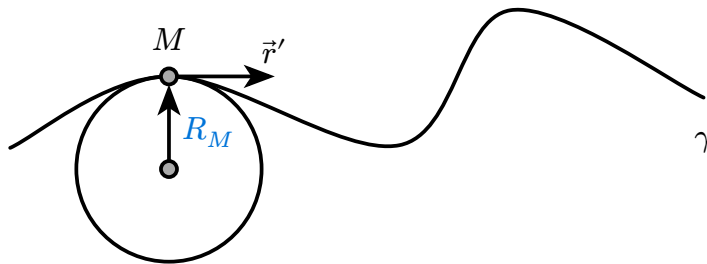
$\angle AOB = \alpha$, $|\overset{\frown}{AB}| = \alpha R$, тогда $K_A = \lim_{B \rightarrow A} \frac{\alpha}{\alpha R} = \frac{1}{R}$

- Радиус кривизны

Def. Величина $\frac{1}{K_A} = R_A$, обратная кривизне в точке, называется радиусом кривизны в точке

Nota. У окружности радиус кривизны совпадает с ее собственным радиусом.

Сама окружность – линия постоянной кривизны. Другая такая линия постоянной кривизны – прямая, где кривизна равна 0



- Центр кривизны

Def. Центр кривизны кривой γ в точке M – это точка на нормали к γ в точке M на расстоянии R_M от M , находящаяся в той полуплоскости, разделенной касательной, что и окрестность кривой

Def. Эволюта кривой – множество центров кривизны

Для характеристик можно выразить другие формулы:

- Кривизна $K = \lim_{\Delta s \rightarrow 0} \frac{\Delta \varphi}{\Delta s} = \varphi'(s) = \frac{d\varphi}{dt} \frac{dt}{ds} \stackrel{t=x}{=} \frac{\frac{d\varphi}{dx}}{\frac{ds}{dx}}$
 $\text{tg } \varphi = \frac{dy}{dx}, \quad \varphi = \arctg \frac{dy}{dx}$ – угол касательной

$$\frac{ds}{dx} = \sqrt{1 + y_x'^2}$$

$$\frac{d\varphi}{dx} = \frac{\frac{d^2y}{dx^2}}{\left(\frac{dy}{dx}\right)^2 + 1}$$

$$K = \frac{\frac{d^2y}{dx^2}}{\left(\left(\frac{dy}{dx}\right)^2 + 1\right)^{\frac{3}{2}}}$$

В параметрическом задании $K = \frac{|y_t''x_t' - y_t'x_t''|}{(x'^2 + y'^2)^{\frac{3}{2}}}$

• Центр кривизны:
$$\begin{cases} x_0 = x - \frac{y'(x'^2 + y'^2)}{x'y'' - x''y'} \\ y_0 = y + \frac{x'(x'^2 + y'^2)}{x'y'' - x''y'} \end{cases}$$

1.7.3 Конструирование кривой по Безье

Во Франции конца 1950-х годов автомобилестроение переживало эпоху расцвета дизайна, и инженерам нужен был способ описывать сложные, плавные изгибы кузовов автомобилей не на чертежной доске, а с помощью первых компьютеров для станков с ЧПУ. Традиционных инструментов для этой задачи было недостаточно

В этот период сразу два французских автогиганта, Citroën и Renault, независимо друг от друга работали над решением этой проблемы

Пьер Безье из Renault, инженер-механик и электрик, пришел к решению к 1962 году, разработав свою систему для компьютерного проектирования кузовов автомобилей, аэродинамические свойства которых легко задаются

Def. Кривая Безье n -ого порядка в параметрической форме описывается функцией

$$B(t) = \sum_{i=0}^n P_i b_{i,n}(t), \text{ где}$$

- $0 \leq t \leq 1$,
- $P_i = (x_i(t), y_i(t))$ – координаты опорных точек,
- а $b_{i,n}(t) = C_n^i t^i (1-t)^{n-i}$ – полиномы Бернштейна, где i – порядковый номер опорной точки

Nota. Упорядоченное множество опорных точек $(P_0, P_1, \dots, P_n)$ называется характеристической ломаной

Nota. Прямое вычисление $B(t)$ как пару функций $B_x(t)$ и $B_y(t)$, определяющих точки кривой, – сложная задача, поэтому чаще используют алгоритм Кастельжо:

Заданы точки $P_0, P_1, \dots, P_n$ нулевого порядка

Тогда $P_i^j(t) = (1-t)P_i^{j-1}(t) + tP_{i+1}^{j-1}(t)$, где i – номер вершины, а j – порядок вершины

Поль де Кастельжо, работая в Citroën, разработал независимо от Безье в 1959 году алгоритм для построения кривой, однако его разработки оставались коммерческой тайной, поэтому кривые получили название в честь Безье

Рассмотрим примеры:

Ех. 1. $n = 1$, даны P_0 и P_1

Для точки первого порядка $P_0^1 = (1-t)P_0^{1-1} + tP_1^{1-1} = (1-t)P_0 + tP_1$



В $t = 0$ $P_0^1 = P_0$, в $t = 1$ $P_0^1 = P_1$. При $0 < t < 1$ $P_0^1(t)$ – точка, пробегающая отрезок

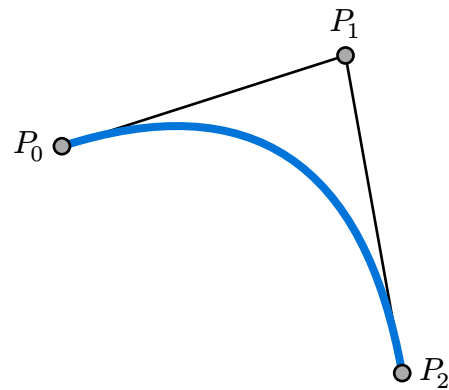
В данном случае (так как меньше двух точек нельзя задать) $P_0^1(t) = B(t)$, то есть при $n = 1$ кривая Безье – отрезок P_0P_1

Ех. 2. $n = 2$, даны P_0, P_1, P_2

По определению $B(t) = \sum_{i=0}^2 P_i C_2^i t^i (1-t)^{2-i} = P_0(1-t)^2 + 2P_1t(1-t) + P_2t^2$

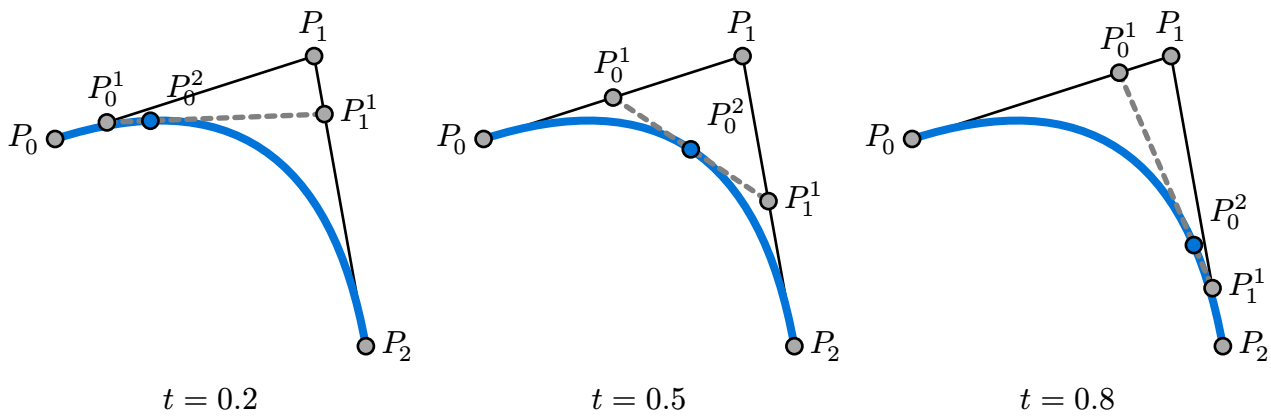
По Кастельжо $P_0(t=0)$, $P_2(t=1)$:

- $P_0^1(t) = (1-t)P_0 + tP_1$
- $P_1^1(t) = (1-t)P_1 + tP_2$
- $P_1^2(t) = (1-t)P_0^1 + tP_1^1$
 $= (1-t)^2P_0 + 2(1-t)tP_1 + t^2P_2$
 $= B(t)$



В случае $n = 2$ кривая Безье – это парабола (в частном случае прямая)

По алгоритму Кастельжо становится видно, что характеристическая ломаная является касательной к кривой Безье. Сами точки первого порядка при разных t дают отрезки, которые касаются кривой, а точка второго порядка лежит непосредственно на кривой Безье:

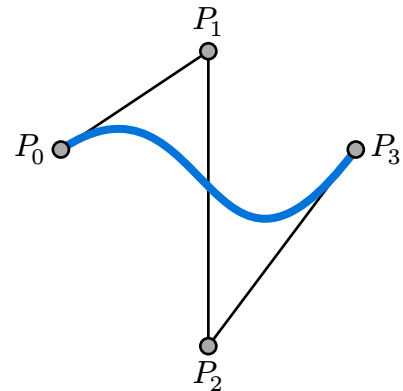


Ех. 3. $n = 3$, даны P_0, P_1, P_2, P_3

$$B(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3$$

Или в матричном виде $B(t) = (t^3, t^2, t, 1) \cdot M_B \cdot \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{pmatrix}$

Здесь $M_B = \begin{pmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$ – матрица Безье



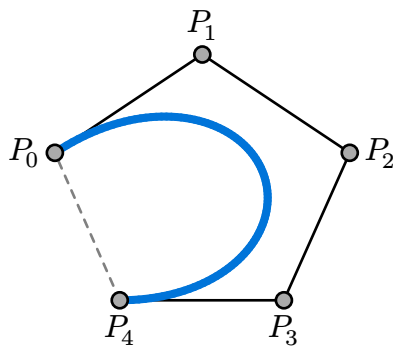
Для кривых Безье справедливы свойства:

- Порядок кривой Безье инвариантен при аффинных преобразованиях

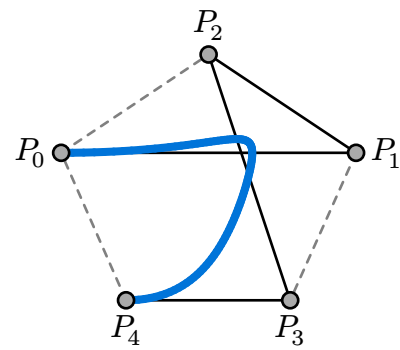
Пусть $P_0, \dots, P_n$ содержит k коллинеарных точек. $\mathcal{F}(P_0, \dots, P_n) = P'_0, \dots, P'_n$ – образы точек при аффинном преобразовании. И так как матрица F не вырождена, среди $P'_0, \dots, P'_n$ останется k коллинеарных, таким образом порядок кривой сохранится

То есть можно не подвергать всю кривую аффинному преобразованию, а всего лишь ее ломаную

- Непрерывность
- Вся кривая лежит внутри многоугольника $P_0, \dots, P_n$ или выпуклой оболочки



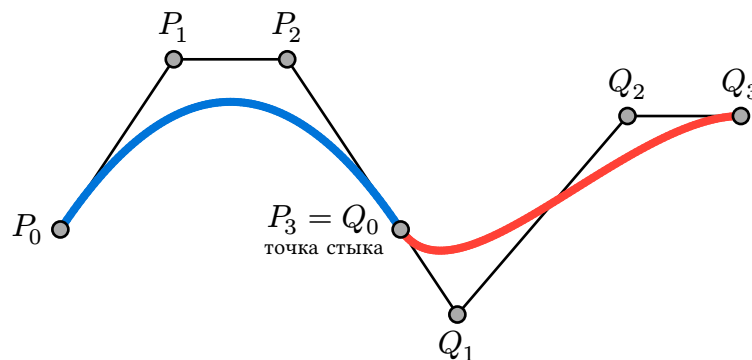
Кривая находится
в пятиугольнике $P_0P_1P_2P_3P_4$



Кривая находится
в пятиугольнике $P_0P_2P_1P_3P_4$

- Симметрия при смене порядка обхода ломаной
- Кусочная гладкость. Польза этого свойства заключается в том, что кривые Безье допускают гладкое сочленение двух кривых

Def. Сплайн – составная кривая, гладкая в точках стыка



Nota. Геометрически, гладкость эквивалентна спрямляемости, то есть в малой окрестности кривая - это почти прямая

Тогда сформулируем правило: чтобы построить сплайн Безье, нужно состыковать кривые $B_1(t)$ и $B_2(t)$ так, что предпоследняя опорная точка P_{n-1} кривой $B_1(t)$, вторая опорная точка Q_1 кривой $B_2(t)$ и точка стыка $P_n = Q_0$ были расположены на одной прямой (то есть коллинеарно)

Nota. Окружность и эллипс нельзя в точности представить кривыми Безье

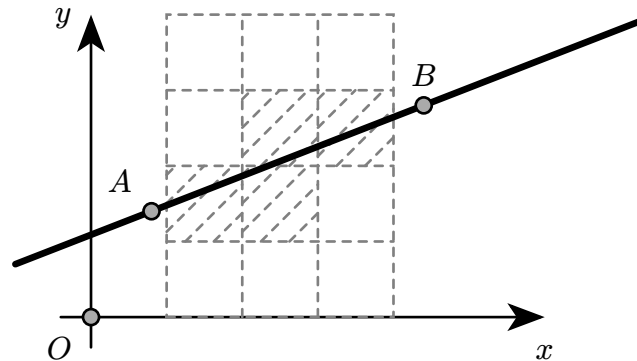
1.7.4 Дискретизация (растровое изображение) линий

Рассмотрим построение отрезка прямой с известным наклоном или проходящей через две данные точки на экране в пикселях

1. Естественный алгоритм

Нам известно уравнение прямой $\frac{x - x_A}{x_B - x_A} = \frac{y - y_A}{y_B - y_A}$

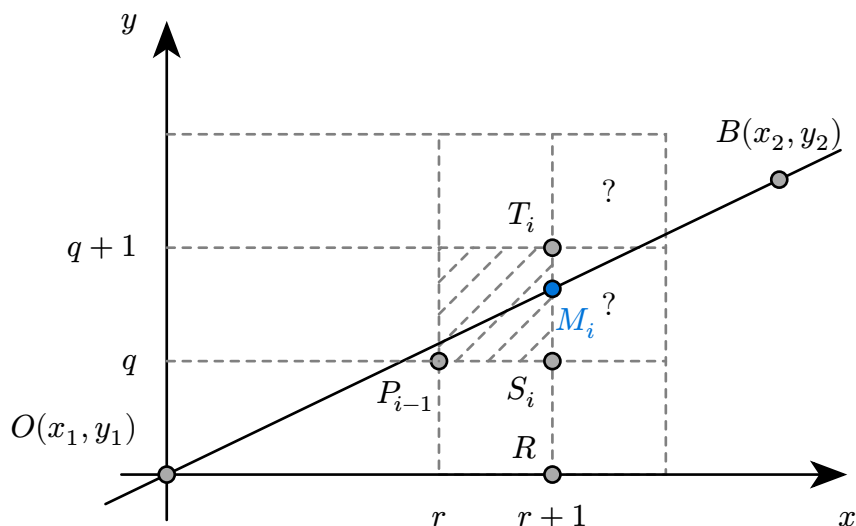
Если выразим y через x , получим $y = y_A + \frac{y_B - y_A}{x_B - x_A}(x - x_A)$ – деление, из-за которого мы вынуждены округлять при выборе пикселя для y



Лучше использовать алгоритм Брезенхэма

2. Алгоритм Брезенхэма

Считаем, что $k \in [0, 1]$, то есть прямая ближе к оси Ox , чем к Oy . При $k \in [1, \infty)$ и $k < 0$ алгоритм аналогичен



Сделаем допущение, что начало координат находится в точке (x_1, y_1) . Пусть на предыдущем шаге мы выбрали пиксель P_{i-1} с координатами (r, q) в системе Oxy . Тогда соседними кандидатами на следующем шаге будут $S_i(r+1, q)$ и $T_i(r+1, q+1)$

Алгоритм сводится к тому, чтобы из двух пикселей S_i и T_i выбрать тот, который ближе к точке M_i

Введем величину $\Delta_i = S - T$, где $S = |S_i - M_i|$, $T = |T_i - M_i|$ – расстояния от точки M_i до соответствующих пикселей

Если $\Delta_i \geq 0$, то есть $S \geq T$, выбираем пиксель T_i , иначе – S_i

Из подобия треугольников получаем пропорцию: $k = \frac{dy}{dx} = \frac{M_i R}{OR} = \frac{q + S}{r + 1}$

Отсюда $S = \frac{dy}{dx}(r + 1) - q$

Так как $S + T = 1$, $T = 1 - S$. Умножая разность $S - T$ на dx , получаем $dx(S - T) = 2(r + 1)dy - (2q + 1)dx$

Если $dx > 0$, знак $\Delta_i = S - T$ совпадает со знаком $dx \cdot \Delta_i$. Обозначим $d_i = dx \cdot \Delta_i$, тогда $d_i = 2(r + 1)dy - (2q + 1)dx$

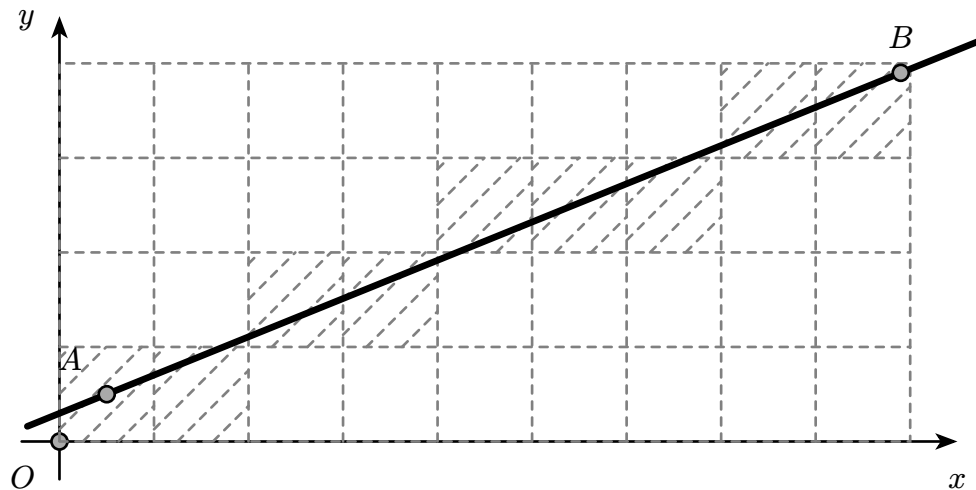
На следующем шаге $(i + 1)$ координаты текущего пикселя будут (x_i, y_i) , тогда $d_{i+1} = 2x_i dy - 2y_i dx + 2dy - dx$

Из этого $d_{i+1} - d_i = \underbrace{2(x_i - x_{i-1})}_{=1} dy - 2(y_i - y_{i-1}) dx$, то есть $d_{i+1} = d_i + 2dy - 2(y_i - y_{i-1}) dx$

Здесь возможны два случая:

- Если $d_i \geq 0$, выбираем пиксель T_i с координатами (x_i, y_i) . Тогда приращение координаты y равно 1, и формула упрощается: $d_{i+1} = d_i + 2(dy - dx)$
- Если $d_i < 0$, выбираем пиксель S_i с координатами (x_i, y_i) , где $y_i = y_{i-1}$, поэтому $d_{i+1} = d_i + 2dy$

Начальное значение d_1 вычисляется для первого шага $i = 1$ как $d_1 = 2dy - dx$



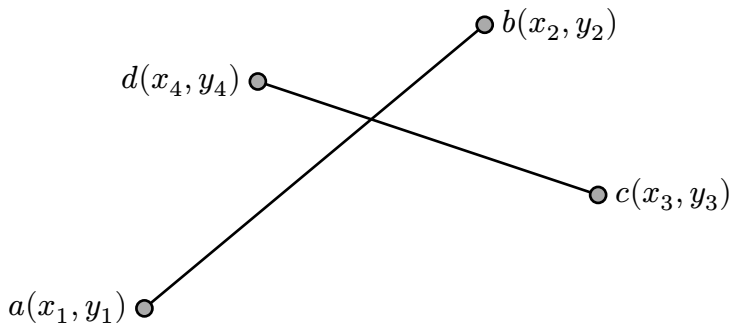
Получаем набор пикселей, которые соединяют центры пикселей, находящихся в концах отрезка, причем этот набор располагается поровну по обе стороны отрезка

2. Геометрический поиск

2.1 Пересечение отрезков

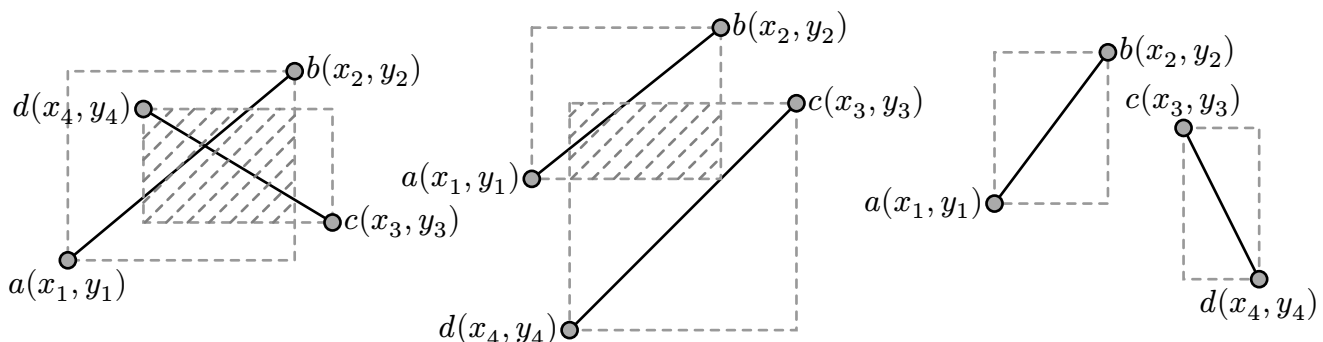
Геометрическая задача – определить, какие из N отрезков на плоскости пересекаются, и найти точки пересечения

Для начала рассмотрим два отрезка:



Отрезки будут заданы координатами вершин $a(x_1, y_1)$, $b(x_2, y_2)$, $c(x_3, y_3)$, $d(x_4, y_4)$

Грубый способ определить то, что отрезки не пересекаются, – сравнить абсциссы $\min(x_1, x_2)$ и $\max(x_3, x_4)$ (или наоборот) или аналогично ординаты (то есть прямоугольники, где диагональ – отрезок)



Как видно, пересечение ограничивающих отрезки прямоугольников является *необходимым* условием для пересечения отрезков, но не *достаточным* (см. второй пример), поэтому применяют другие алгоритмы

Из школьной геометрии знаем, что любая прямая делит плоскость на две полуплоскости

Если отрезки пересекаются, то точки c и d лежат в разных полуплоскостях относительно прямой l (обратное при этом неверно)

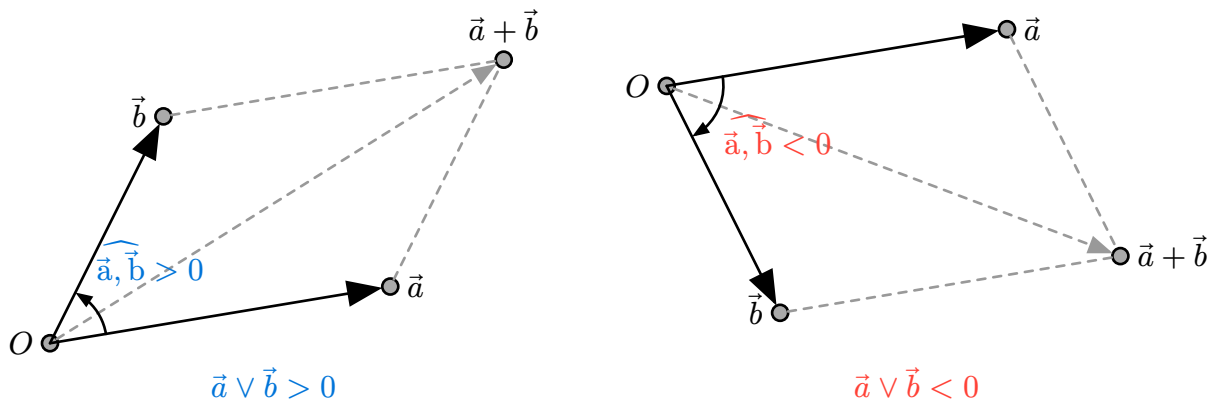
Однако аналогично, если отрезки пересекаются, то точки a и b лежат в разных полуплоскостях относительно прямой p , образованной точками c и d

Теперь нужен алгоритм, который проверяет, что точки одного отрезка лежат по разные стороны относительно прямой другого отрезка

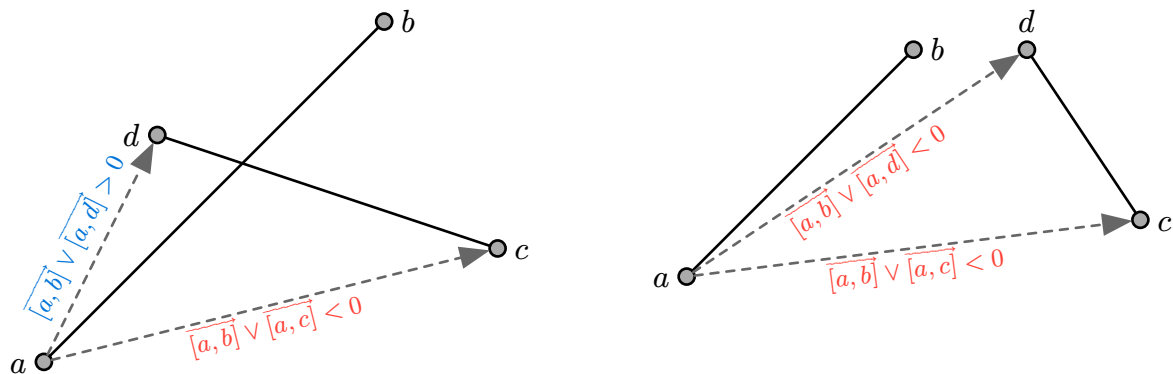
Мет. Пусть $\vec{a}, \vec{b}$ находятся на плоскости. Косое произведение $\vec{a} \vee \vec{b} \stackrel{\text{def}}{=} |\vec{a}| |\vec{b}| \sin(\widehat{\vec{a}, \vec{b}}) =$

$$|\vec{a} \times \vec{b}| = \pm \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ a_1 & a_2 & 0 \\ b_1 & b_2 & 0 \end{vmatrix} = \pm \begin{vmatrix} a_1 & a_2 \\ b_1 & b_2 \end{vmatrix} \vec{k} = \pm \begin{vmatrix} a_1 & a_2 \\ b_1 & b_2 \end{vmatrix} = \begin{vmatrix} a_1 & a_2 \\ b_1 & b_2 \end{vmatrix} - \text{площадь параллелограмма}$$

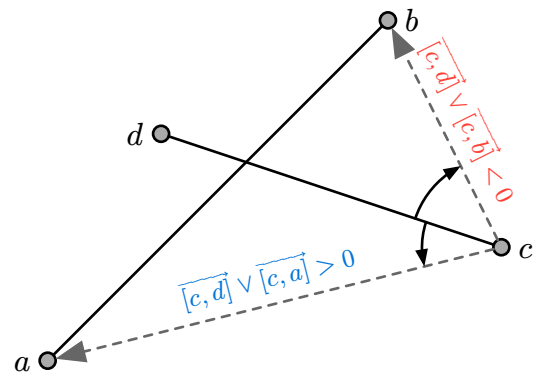
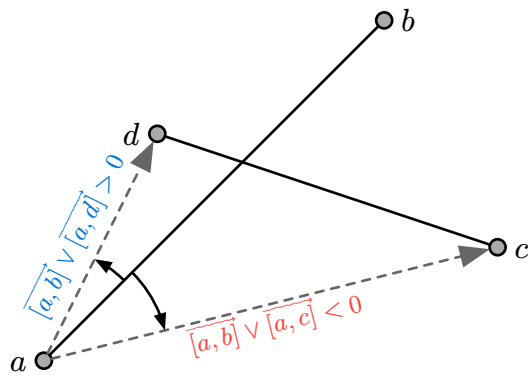
Косое произведение со знаком «плюс» равно площади параллелограмма, образованными векторами, концы которых расположены по часовой стрелке



Применим к задаче. Косое произведение показывает ориентацию, поэтому, если точки находятся по разные стороны от прямой, то косые произведения $\overrightarrow{[a, b]} \vee \overrightarrow{[a, c]}$ и $\overrightarrow{[a, b]} \vee \overrightarrow{[a, d]}$ должны иметь разные знаки

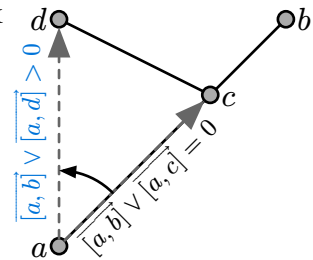


Аналогично проверяем произведения $\overrightarrow{[c, d]} \vee \overrightarrow{[c, a]}$ и $\overrightarrow{[c, d]} \vee \overrightarrow{[c, b]}$



Для непересекающихся отрезков одна пара произведений из двух будет иметь один знак

Nota. Если одна из точек одного отрезка находится на другом отрезке, то косое произведение равно 0



Теперь найдем саму точку пересечения. Нам известны уравнения прямых:

$$\begin{vmatrix} x - x_1 & y - y_1 \\ x_2 - x_1 & y_2 - y_1 \end{vmatrix} = 0$$

$$\begin{vmatrix} x - x_3 & y - y_3 \\ x_4 - x_3 & y_4 - y_3 \end{vmatrix} = 0$$

Далее решаем систему уравнений:

$$\begin{cases} \begin{vmatrix} x - x_1 & y - y_1 \\ x_2 - x_1 & y_2 - y_1 \end{vmatrix} = 0 \\ \begin{vmatrix} x - x_3 & y - y_3 \\ x_4 - x_3 & y_4 - y_3 \end{vmatrix} = 0 \end{cases}$$

$$\text{Получаем } \begin{cases} k_1 = \frac{y_2 - y_1}{x_2 - x_1} \\ k_2 = \frac{y_4 - y_3}{x_4 - x_3} \\ b_1 = y_1 - k_1 x_1 \\ b_2 = y_3 - k_2 x_3 \\ x = \frac{b_2 - b_1}{k_1 - k_2} \\ y = k_1 x + b_1 \end{cases} \Leftrightarrow \begin{cases} x = \frac{(y_3 - \frac{y_4 - y_3}{x_4 - x_3} x_3) - (y_1 - \frac{y_2 - y_1}{x_2 - x_1} x_1)}{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_4 - y_3}{x_4 - x_3}} \\ y = k_1 x + b_1 \end{cases}$$

Результатом будет точка пересечения прямых. Так как прямые бесконечно длинные, то нужна дополнительная проверка, описанная выше, на то, что пересекаются действительно отрезки, а не только прямые

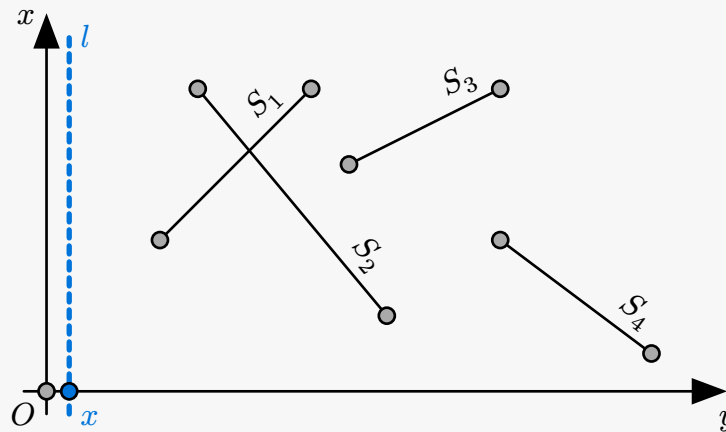
Nota. Если число отрезков велико, то проверка попарных пересечений очень затратна, асимптотическая сложность – $O(n^2)$

Поэтому применяют алгоритм «заметающей» прямой (или алгоритм Бентли-Оттмана):

Сделаем допущение, что точки пересечения образуются пересечением только двух отрезков и нет вертикальных отрезков

Тогда:

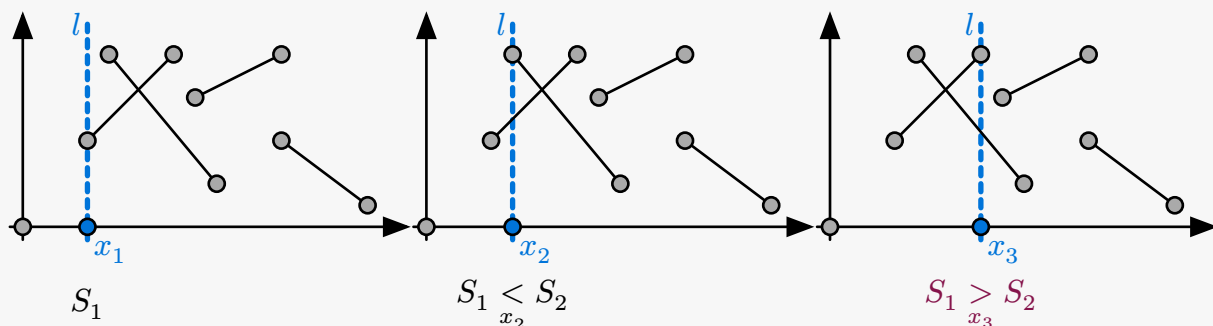
1. Берем вертикальную прямую l , проходящую через точку $(0, x)$
2. Все отрезки заданы списком координат их концов. Тогда можем отсортировать этот список по возрастанию абсцисс точек



3. Прямая l движется направо и пересекает концы отрезков. Встречу прямой с концами отрезков обозначим событием. Все отрезки, отвечающие событию x (точка с абсциссой x принадлежит этим отрезкам), называется смежными по x

Далее вводим отношение сравнения отрезков S_i и S_j : $S_i >_x S_j$ (S_i выше S_j) означает, что в точке x ордината точки $S_i \cap l$ больше ординаты $S_j \cap l$

4. Если найдена смена отношения $>$ в разных событиях, то отрезки пересеклись

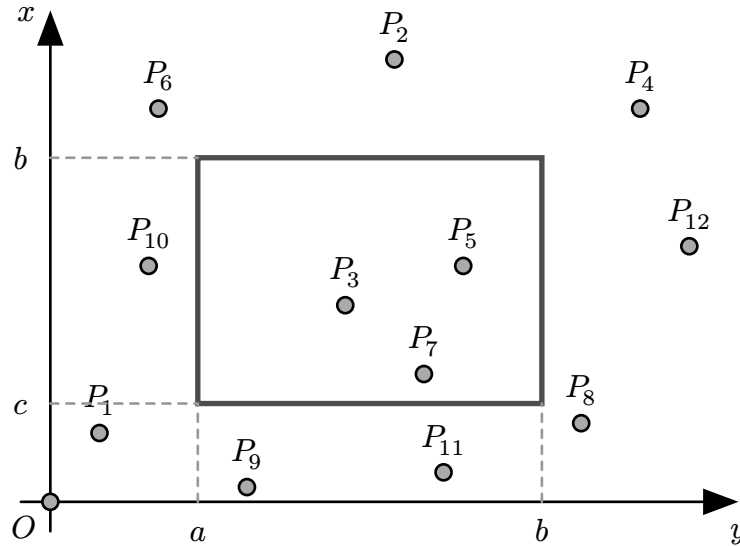


Сложность данного алгоритма составляет $O((n + k) \log n)$, где n – число отрезков, а k – число пересечений

2.2 Локализация точки в многоугольнике

2.2.1 Локализация точки в прямоугольнике

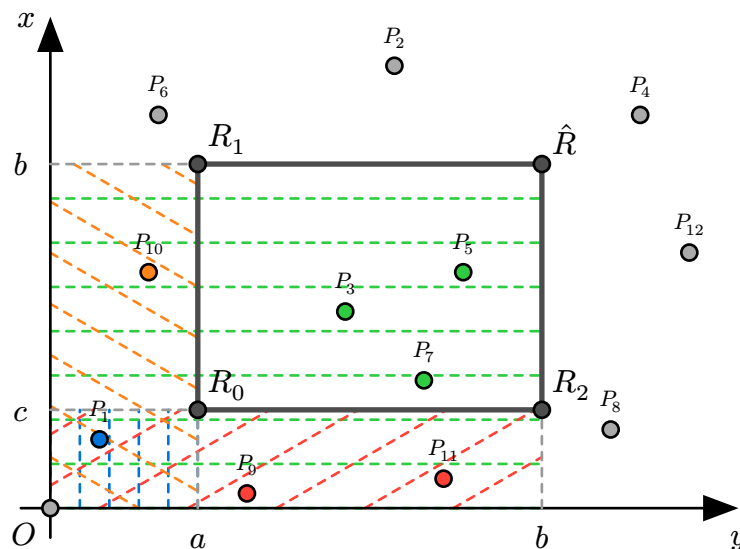
Задача состоит в том, чтобы понять, находится ли точка на плоскости внутри прямоугольника, стороны которого ориентированы параллельно координатным осям



Для точки $P_i(x, y)$ принадлежность определяется условиями $\begin{cases} a \leq x \leq b \\ c \leq y \leq d \end{cases}$

Можно упорядочить процесс для определения принадлежности P_i ячейки прямоугольной сетки. Вводим отношение – «точка M доминирует над точкой N » или $N^M \Leftrightarrow \begin{cases} x_N \leq x_M \\ y_N \leq y_M \end{cases}$

Тогда найдем в прямоугольнике доминирующую вершину $\hat{R}$ (правый верхний угол). Обозначим $M(R)$ как множество точек, доминируемых вершиной R , а $Q(R)$ как количество этих точек



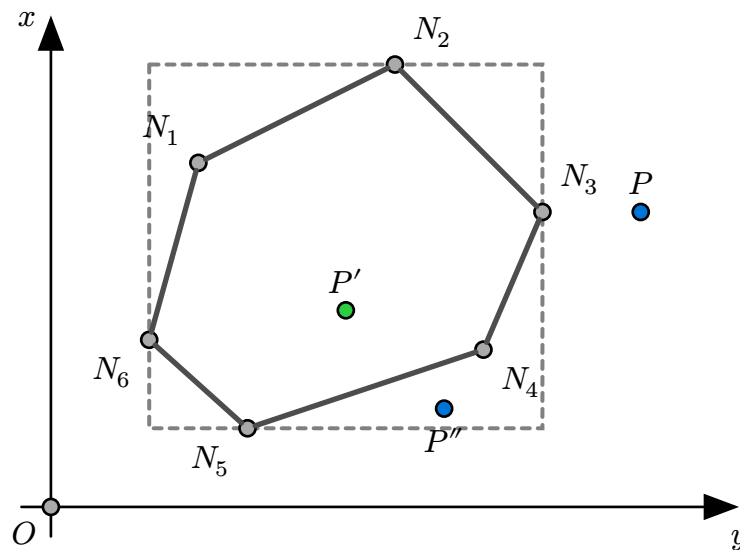
Тогда, чтобы найти, сколько точек находятся в прямоугольнике, воспользуемся формулой $Q(\hat{R}) - Q(R_1) - Q(R_2) + Q(R_0)$. А само множество задается как $M(\hat{R}) - M(R_1) - M(R_2) + M(R_0)$

2.2.2 Локализация точки в n -угольнике

Есть несколько методов:

1. Габаритный метод

Пусть есть выпуклый n -угольник $\mathcal{N}$, нужно понять, находится ли точка внутри многоугольника

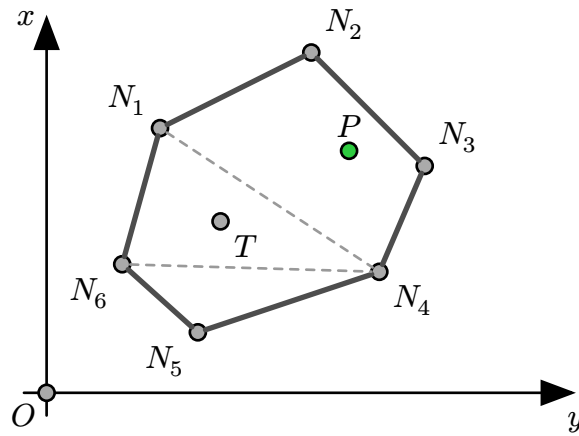


Необходимым условием принадлежности точки внутри многоугольника – принадлежность точки внутри прямоугольника, которые описывает многоугольник. То есть можно проверить условия $x_P < x_{\min}$, $x_P > x_{\max}$, $y_P < y_{\min}$, $y_P > y_{\max}$

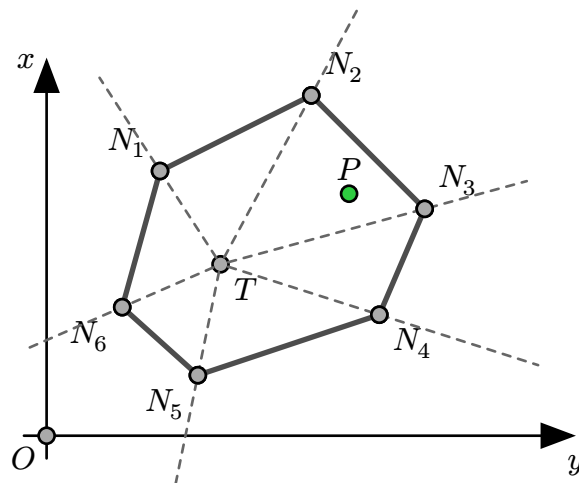
Такой метод работает всего лишь для отсева не принадлежащих точек, но не поможет различить принадлежность точек P' и P''

2. Угловой метод

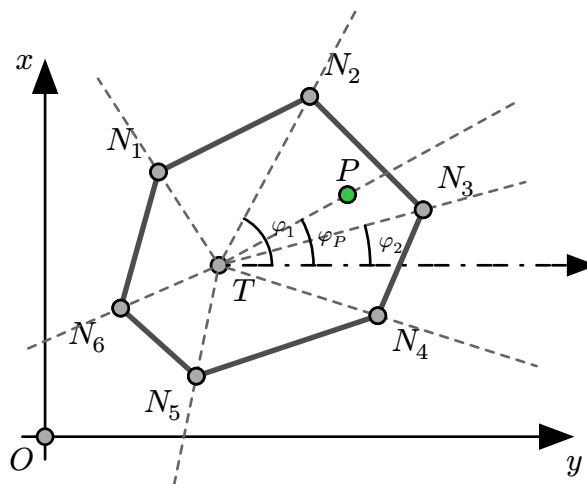
Возьмем произвольную точку T внутри выпуклого многоугольника, например, центр тяжести одного из вписанного треугольника или угодно другую, которую легко определить



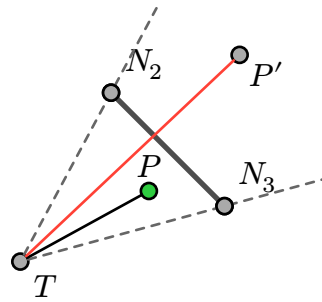
Из этой точки выпускаем лучи TN_i и луч TP



В выпуклом многоугольнике углы между лучами к вершинам и осью Ox будут упорядочены (такое не работает для невыпуклого многоугольника), поэтому, записав порядок углов для вершин N_i , находим угол φ_P для луча TP и углы φ_i, φ_{i+1} такие, что $\varphi_i \leq \varphi_P \leq \varphi_{i+1}$

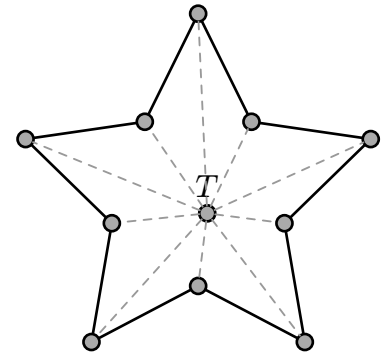


Тогда точка P лежит в секторе N_iTN_{i+1} . Далее с помощью косого произведения определяем, лежит ли точка P в треугольнике N_iTN_{i+1} , то не пересекаются ли отрезки TP и N_iN_{i+1}



В звездчатом многоугольнике такой метод также работает, если выбрать точку в центре, но в произвольном невыпуклом (особенно с самопересечениями) такое не работает

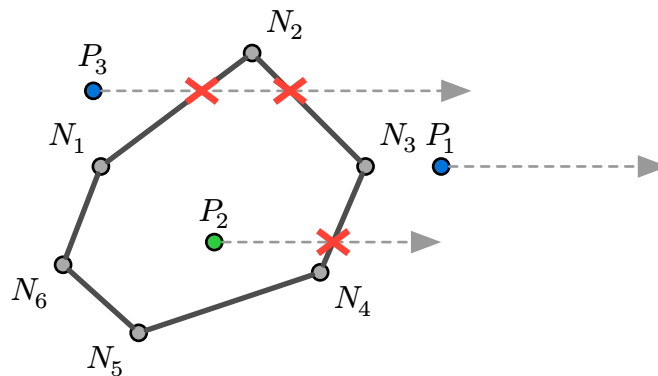
Если работа с углами представляет неудобства (так как синусы большинства углов – величины трансцендентные), то можно сравнивать ориентированные площади



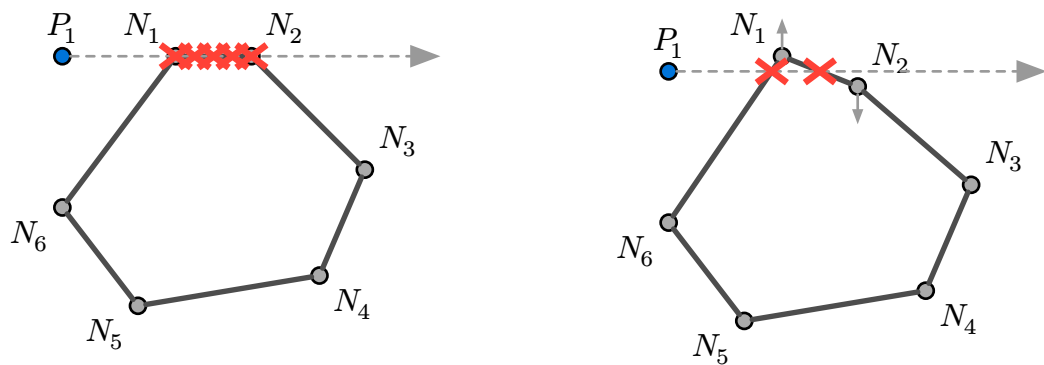
Ориентированная площадь имеет знак в зависимости от направления обхода ломаной, поэтому, если площади треугольников $\triangle PTN_i$, $\triangle PN_iN_{i+1}$ и $\triangle PN_{i+1}T$ одних знаков, то точка находится внутри $\triangle TNN_{i+1}$

3. Лучевой метод

Лучевой метод заключается в выпускании луча из данной точки в произвольном направлении (например, в направлении Ox). Считаем, что если луч пересек ребра выпуклого многоугольника нечетное число раз (то есть ровно один), то точка находится внутри многоугольника



Здесь есть случай с подвохом: луч пересекает какую-либо вершину многоугольника (в том числе какое-либо ребро). В этом случае ординату вершины уменьшаем или увеличиваем на бесконечно малую величину:



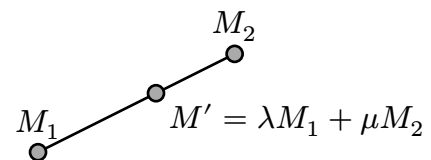
2.3 Выпуклые оболочки

Рассмотрим точечное множество $M = \{M_1, \dots, M_n\}$

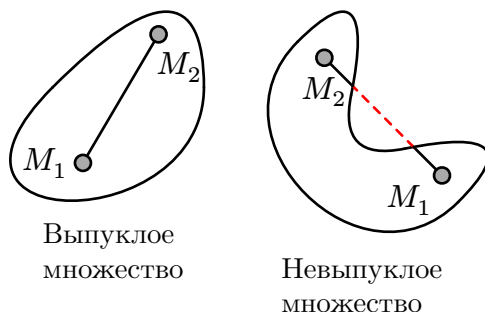
Мет. Выпуклое множество – такое множество, в котором для любых $M_i, M_j \in M$ отрезок $M_i M_j$ тоже принадлежит множеству M

Nota. Также выпуклое множество – множество, где линейная (или аффинная) комбинация любых точек принадлежит множеству: $\forall M_{i_1}, \dots, M_{i_k} \in M$

$$\sum_{j=1}^k \lambda_j M_{i_j} \in M \text{ для всех } \lambda_i \geq 0 \text{ таких, что } \sum_{i=1}^k \lambda_i = 1$$



Ех.

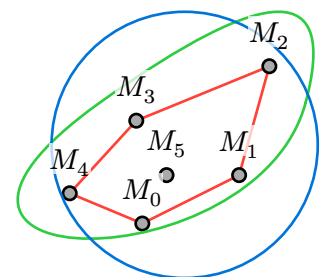


Выпуклое
множество

Невыпуклое
множество

Для данного точечного набора существуют различные выпуклые множества, содержащие набор точек

Def. Наименьшее из выпуклых множеств V , содержащих данное множество M , называется выпуклой оболочкой $\text{conv } M$ множества M

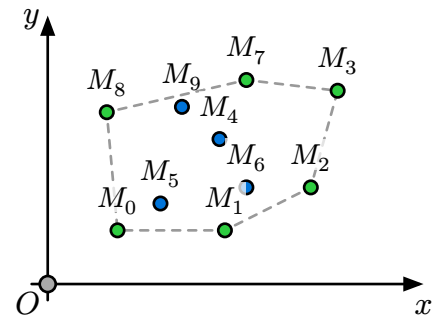


- Nota.*
1. Выпуклая оболочка пустого множества – пустое множество: $\text{conv } \emptyset = \emptyset$
 2. Выпуклая оболочка множества из точки – это множество из точки:
 $\text{conv}\{M\} = \{M\}$
 3. Выпуклая оболочка множества из двух точек – это отрезок: $\text{conv}\{M_1, M_2\} = M_1 M_2$

4. Выпуклая оболочка в общем случае $\text{conv} \{M_i\}_{i=1}^n$ – это выпуклый многоугольник
5. Выпуклая оболочка выпуклого многоугольника – сам многоугольник

Выделим крайние и внутренние точки множества $M = \{M_i\}_{i=1}^n$: крайние точки – это те точки, что являются вершинами многоугольника, остальные считаем внутренними

Если найти все крайние, тогда построение $\text{conv} M$ сводится к их упорядочиванию этих точек и построению ребер

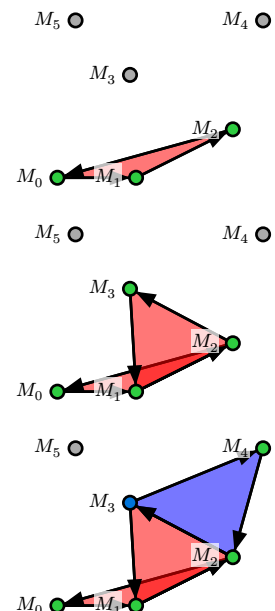


Критерий. Точка M не является крайней (то есть является внутренней) тогда и только тогда, когда существует треугольник $\triangle M_i M_j M_k$, для которого $M \in \triangle M_i M_j M_k$ и $M \neq M_i, M_j, M_k$

Действуя по критерию можно найти все внутренние точки, но алгоритм является очень затратным

Рассмотрим такую ситуацию:

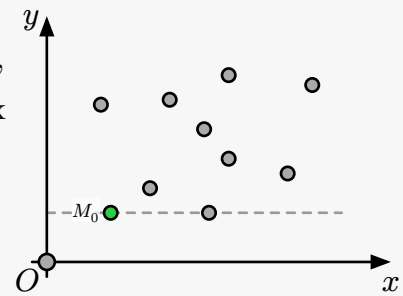
1. Проверяем ориентацию $\triangle M_0 M_1 M_2$, например, с помощью ориентированной площади $S_{\triangle} = \frac{1}{2} \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}$
2. Если треугольник $\triangle M_0 M_1 M_2$ ориентирован против часовой стрелки, то переходим к $\triangle M_1 M_2 M_3$
3. Если треугольник $\triangle M_1 M_2 M_3$ ориентирован против часовой стрелки, то переходим к $\triangle M_2 M_3 M_4$
4. Если треугольник $\triangle M_2 M_3 M_4$ ориентирован в другую сторону, то считаем M_3 внутренней и рассматриваем треугольник $\triangle M_2 M_4 M_5$



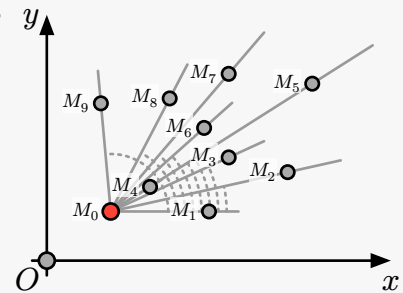
Итак, алгоритм Грэхема:

Дано $M = \{M_i\}_{i=1}^n$ в экранной системе координат

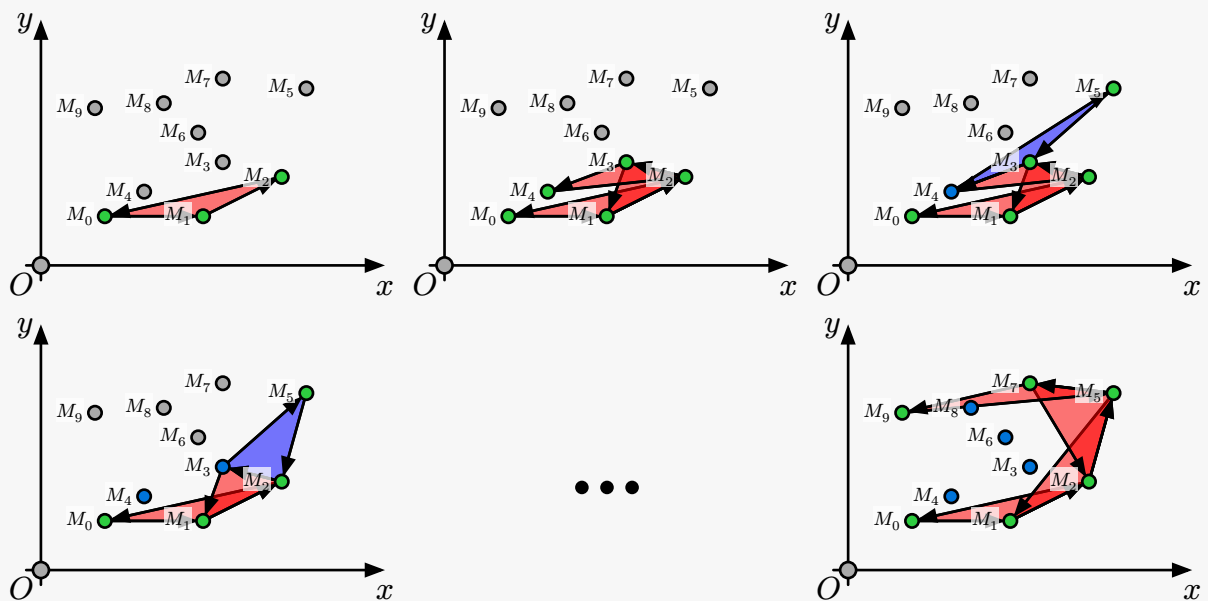
1. Отбираем точки $M_{\min y}$ из M так что $y_0 = \min_{i=1..n} y_i$, далее находим точку M_0 из $M_{\min y}$ такую, что $x_0 = \min_{i=1..n} x_i$ (она будет самой левой из самых нижних и являться крайней)



2. Далее упорядочиваем точки M_i по величине угла в M_0 между каждой точкой M_i и осью Ox



3. Определяем ориентацию троек $M_i M_{i+1} M_{i+2}$. Если точки идут по часовой стрелке, то исключаем M_{i+1} , повторяем с $M_{i-1} M_i M_{i+2}$ до тех пор, пока точки будут идти против часовой стрелки



Такой алгоритм из-за сортировки углов работает за $O(n \log n)$, где $n = |M|$ – число точек во множестве

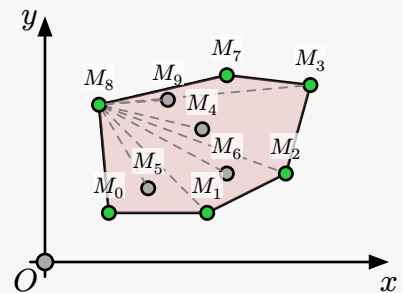
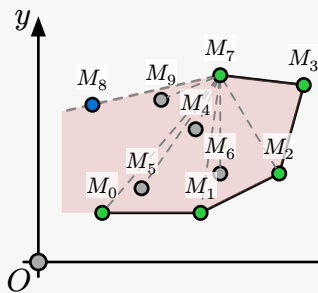
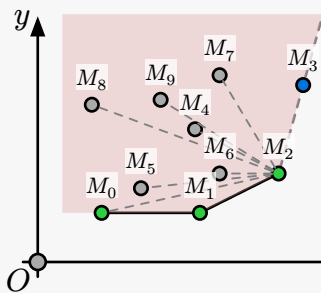
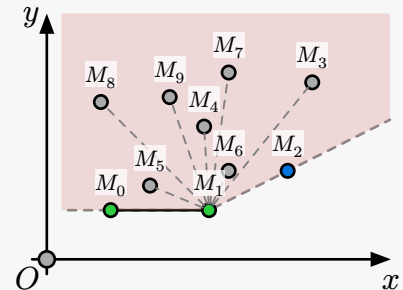
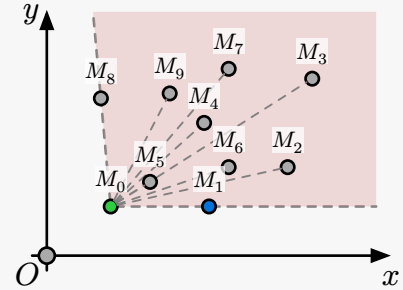
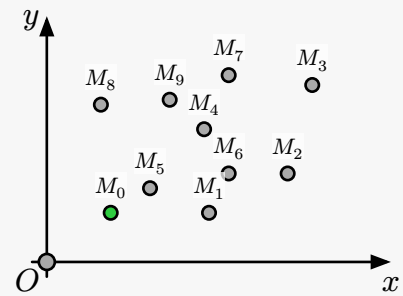
Другой метод, алгоритм Джарвиса (или алгоритм заворачивания подарка), работает так:

Дано $M = \{M_i\}_{i=1}^n$ в экранной системе координат

1. Находим самую левую точку M_0 из самых нижних (или другую такую, чтобы она была крайней)

2. Относительно нее находим ту точку M_1 , которая будет составлять минимальный угол между лучами M_0M_1 и осью Ox . Такая точка M_1 будет являться крайней

3. Повторяем до тех пор, пока не придем к M_0



Алгоритм Джарвиса имеет сложность $O(hn)$, где h – число вершин выпуклой оболочки (в данном примере – 6), а $n = |M|$ – число точек во множестве, поэтому он быстрее алгоритма Грэхема в тех случаях, когда $h < \log n$

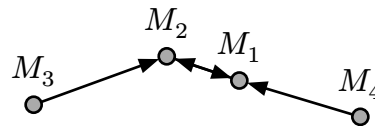
2.4 Близость

Пусть есть задача найти в множестве $M = \{M_1, M_2, \dots, M_n\}$ к данной точке M_k ближайшей точки, ближайшей пары или ближайших соседей

Def. Точка M^* – ближайшая к M_k , если $\text{dist}(M^*, M_k) = \min_j \text{dist}(M_j, M_k)$

Nota. Отношение близости к M_k несимметрично, то есть M^* – ближайшая к $M_k \not\Rightarrow M_k$ – ближайшая к M^*

Ex. M_2 – ближайшая к M_3 , но ближайшая к M_2 не M_3 , а M_1



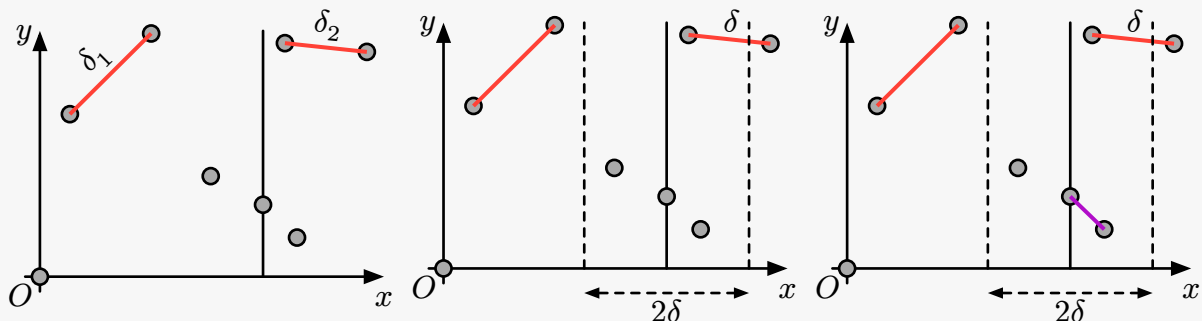
Def. Если отношение близости для пары M_i и M_j симметрично, то (M_i, M_j) – ближайшая пара

2.4.1 Ближайшая пара

Nota. Наивным алгоритмом будет являться поиск всех расстояний $\text{dist}(M_i, M_j)$ для $i \neq j$ и поиск $\min_{i,j} \text{dist}(M_i, M_j)$. Такой алгоритм обладает сложностью $O(n^2)$

Рассмотрим алгоритм «разделяй и властвуй»

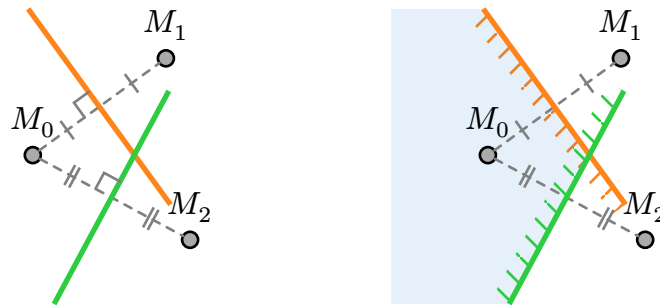
1. Разделяем M на левое подмножество M_1 и правое M_2
2. Пусть нашлись ближайшие пары $(L_1, L_2) \in M_1$ и $(P_1, P_2) \in M_2$
Для них $\delta_1 = \text{dist}(L_1, L_2)$ и $\delta_2 = \text{dist}(P_1, P_2)$. Обозначим $\delta = \min(\delta_1, \delta_2)$
3. В δ -полосе (полосе шириной 2δ) разделителя находим ближайшую пару, если она там есть, и проверяем, что расстояние для пары меньше, чем δ
4. Повторяем алгоритм рекурсивно. Концом деления можно считать тот момент, когда в подмножестве остается 2 или 3 точки



2.4.2 Построение локус-сети (диаграммы Вороного) множества

Def. Локус точки M_0 – множество точек $N(x, y)$ такие, что $\text{dist}(M_0, N) < \text{dist}(M_i, N)$ для $i = 1, 2, \dots$ (то есть множество точек, которые к M_0 ближе, чем к остальным)

Nota. Форма локуса – это пересечение полуплоскостей, каждая из которых задается неравенством $\text{dist}(M_0, N) < \text{dist}(M_i, N)$ и ограничена серединным перпендикуляром к отрезку M_0M_i



Nota. Точки локуса для M_0 лежат в той же полуплоскости, что и M_0 по отношению к серединному перпендикуляру отрезка M_0M_1

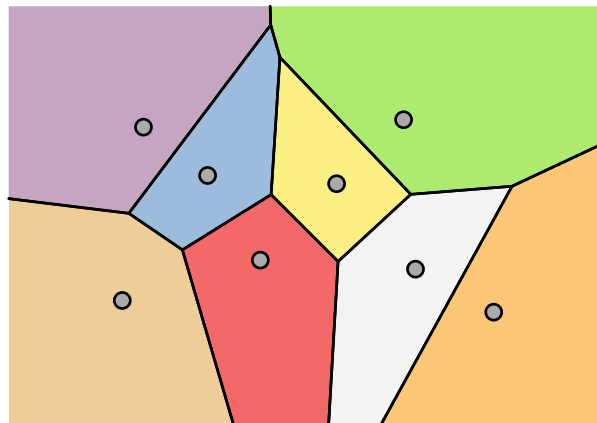
Если M_0 – внутренняя точка множества M , то ее локус – это замкнутый многоугольник. Если M_0 принадлежит выпуклой оболочке, то локус – это неограниченный полигон

Nota. Наложим требование на M : никакие 4 точки не лежат на одной окружности

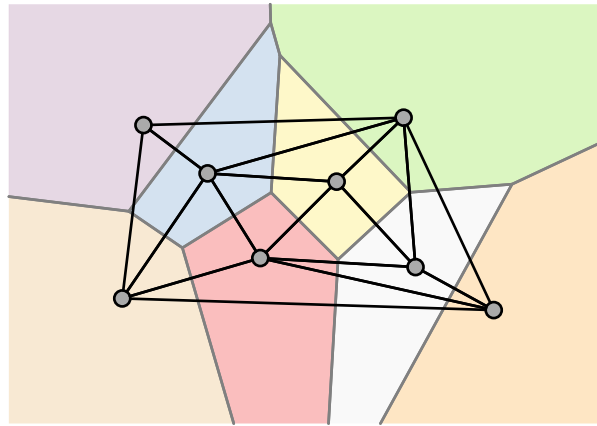
Тогда у локус-сети есть свойства:

1. Любая вершина – пересечение трех ребер
2. Любое ребро связывает 2 вершины
3. Любая вершина – центр окружности, описанной около трех точек M
4. Любое ребро определяет ближайшего соседа

Итак, диаграмма Вороного выглядит так:



Def. Граф, двойственный диаграмме Вороного, задает триангуляцию M (в частности триангуляцию Делоне):



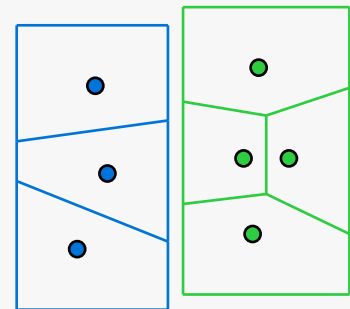
Диаграмму Вороного можно описать набором вершин, в которых сходятся ребра, и набор ребер для каждой вершины. Триангуляция представляется набором треугольных ячеек

С помощью диаграммы Вороного задача о нахождении ближайшей точки решается легче – можно рассматривать лишь точки в окружении

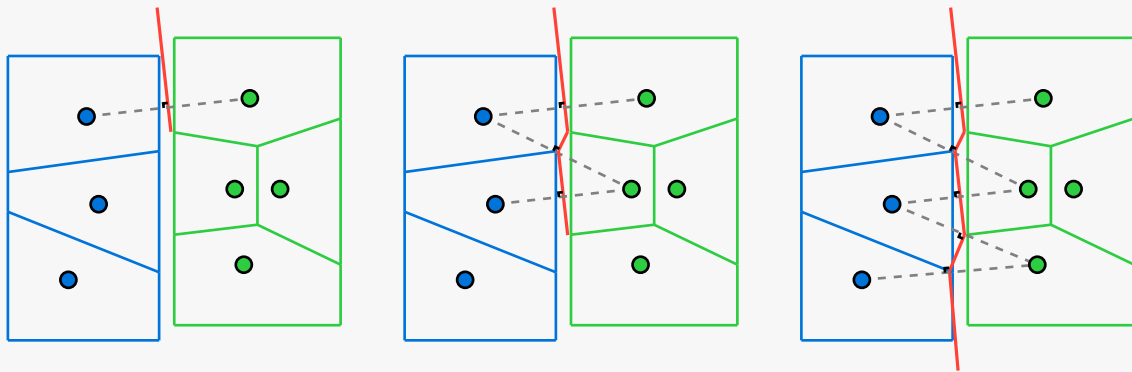
Диаграмму Вороного можно построить:

- По наивному алгоритму с помощью пересечения полуплоскостей, образованных серединными перпендикулярами
- По алгоритму «разделяй и властвуй»

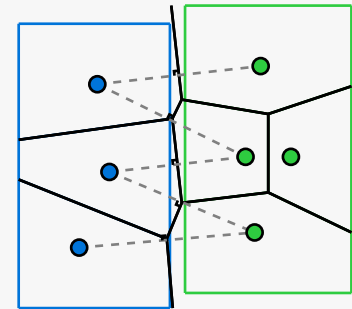
1. Разделяем точки на две половины по x , рекурсивно строим диаграммы для левой половины L и правой R



2. Строим разделяющую ломаную – множество точек, равноудалённых от ближайших точек левой и правой диаграммы. Ломаная состоит из отрезков серединных перпендикуляров между парами (L_i, R_i) , где $L_i \in L$, $R_i \in R$
 - Находим верхнюю пару (L_0, R_0) (самый верхний луч)
 - Двигаемся вниз: на каждом шаге текущий перпендикуляр $L_i R_i$ пересекается с ребром левой или правой диаграмм; переходим в соседнюю ячейку
 - Добавляем отрезки до пересечения
 - Заканчиваем у нижней границы

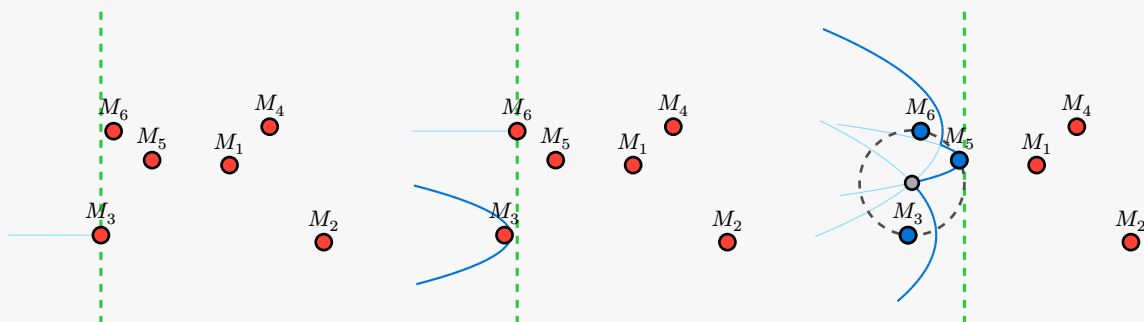


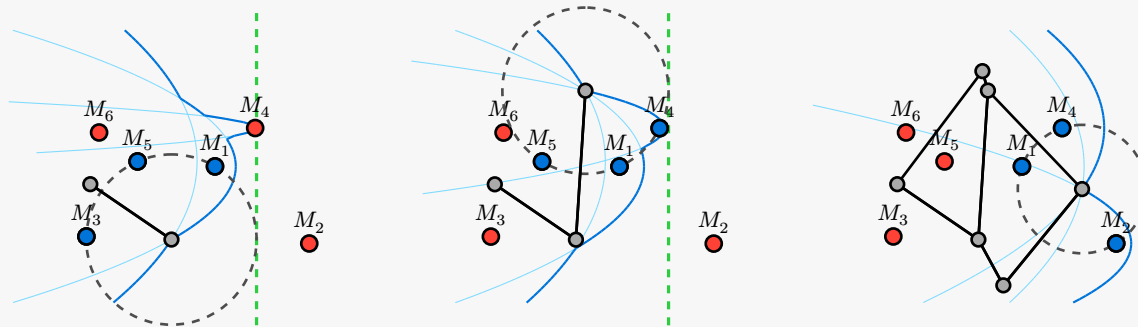
3. Добавляем новую ломаную как ребра между диаграммами, повторяем рекурсивно



- По алгоритму Форчуна (или алгоритм «береговой линий»), где используется заметающая прямая

1. Заметающая прямая (вертикальная или горизонтальная) движется слева направо
2. Когда прямая достигает очередной точки в береговую линию добавляется парабола, образованная заметающей прямой и этой точкой как фокусом
Точка пересечения двух парабол является равноудаленной от фокусом – такая точка будет лежать на ребре диаграммы Вороного
3. Три параболы пересекаются в одной точке, если эта точка равноудаленна от фокусов парабол (то есть фокусы лежат на окружность). Эта точка будет вершиной диаграммы Вороного





У вершин диаграммы, имеющих 2 ребра (такие находятся на границе), третье ребро строится как серединный перпендикуляр двух точек